

D5_Història_i_introducció_al_NPL

February 24, 2026

#D5. Història i introducció al NPL

El Processament del Llenguatge Natural (NLP, per les seves sigles en anglès) és un camp que agrupa la lingüística, la informàtica i la intel·ligència artificial.

El seu objectiu és fer que les màquines **compreguin, interpretin i generin** llenguatge humà.

El text és, avui dia, una de les formes de dades més abundants del món: pàgines web, missatges de correu electrònic, publicacions a les xarxes socials, documents, etc. Totes aquestes fonts generen text no estructurat, cosa que suposa un gran repte per al seu processament automàtic, però alhora obre un ampli ventall d'oportunitats per extreure coneixement i desenvolupar aplicacions basades en el llenguatge natural.

##Evolució històrica del NLP

Regles manuals (1950-1990) * **1950 Test de Turing:** punt de partida de la intel·ligència artificial moderna. * **1966 ELIZA (MIT):** primer chatbot de la història; simulava una conversa entre pacient i terapeuta utilitzant regles escrites manualment. Simulador disponible: <https://www.masswerk.at/elizabot/> * **1980 SHRDL:** sistema pioner de comprensió del llenguatge natural capaç d'interpretar ordres i manipular objectes en un món virtual de blocs. * **1990 Mon-key Island:** exemple lúdic d'ús de parsers i diàlegs predefinitos en videojocs, mostrant com les regles lingüístiques podien aplicar-se a la interacció amb usuaris. Evolució a Maniac Mansion.

Models basats en freqüències i probabilitats (1990-2012) * **1990 Corpus Penn Treebank:** base per a l'entrenament de parsers estadístics. * **1997 LSTM (Hochreiter & Schmidhuber):** introdueix memòria a llarg termini en xarxes neuronals. * **2002 Google News Corpus:** corpus massiu per a models de traducció automàtica. * **2011 · IBM Watson:** guanya el concurs Jeopardy! davant competidors humans.

Embeddings (2013-2016) * **2013 Word2Vec (Google, Mikolov):** converteix paraules en números (vectors) que capturen significats semblants. Exemple: rei - home + dona = reina. * **2014 Glove (Stanford):** millora els vectors mirant tota la frase sencera (no només veïns propers) per entendre millor el context global. * **2016 FastText (Facebook):** descompon paraules en trossos petits (com “joc” = “jo” + “oc”) per entendre paraules noves o amb faltes.

Transformers i LLMs (2017-avui) * **2017 “Attention Is All You Need” (Google) :** inventa els Transformers: en comptes de llegir text seqüencialment, “mira tot d'un cop” per entendre quines paraules són importants entre si. * **2018 BERT (Google):** llegeix en ambdós sentits com nosaltres: entén el context abans i després de cada paraula. Primer model massiu amb preentrenament general (aprèn llenguatge) i fine-tuning específic (s'ajusta a la teva tasca). * **2018 GPT-1 (OpenAI):** prediu la paraula següent (autoregressiu). Aprèn a “continuar” frases com ho faria un humà, generant text coherent. * **2021 GitHub Copilot:** autocompleta codi. Llegeix el

teu codi i suggereix funcions completes. * **2023-avui:** GPT-4 (multimodal), Claude (llarg context), Gemini (Google), Llama 3 (Meta, obert i gratuït), Mistral (europeu, obert). Models multimodals: text + imatge + àudio + vídeo.

El NLP és ara una competència essencial per a qualsevol developer.

##ELIZA: el primer chatbot

Vídeo sobre la història de ELIZA: <https://www.youtube.com/watch?v=zhxNI7V2IzM>

Fem una petita simulació d'ELIZA amb Python. Fes-hi un cop d'ull i intenta deduir les limitacions:

```
[1]: import re

ELIZA_REGLES = [
    (r".*em sento (.+)",      "Per què et sents {}?" ),
    (r".*estic (.+)",        "Fa molt que estàs {}?" ),
    (r".*no puc (.+)",        "Què t'impedeix {}?" ),
    (r".*vull (.+)",          "Per què vols {}?" ),
    (r".*la meva (.+) (.+)",  "Parla'm més de la teva {} {}." ),
    (r".*sí.*",               "Interessant. Continua." ),
    (r".*no.*",               "Per què no?" ),
    (r".*",                   "Entenc. Continua, t'escolto." ),
]

def eliza(entrada):
    entrada = entrada.lower().strip()
    for patro, resposta in ELIZA_REGLES:
        if (m := re.match(patro, entrada)):
            try:
                return resposta.format(*m.groups())
            except:
                return resposta
    return "Entenc. Continua, t'escolto."

print("ELIZA: Hola, sóc ELIZA. T'escolto i si en algun moment vols acabar la_
↳conversa escriu 'sortir'\n")

while (entrada := input("Tu: ")) and entrada.lower().strip() != "sortir":
    print(f"ELIZA: {eliza(entrada)}\n")

print("ELIZA: Fins aviat!")
```

ELIZA: Hola, sóc ELIZA. T'escolto i si en algun moment vols acabar la conversa escriu 'sortir'

Tu: sortir

ELIZA: Fins aviat!

Els chatbots basats en regles com ELIZA depenen de patrons sintàctics rígids. Per exemple, si

canvies una sola paraula d'una frase que no encaixa exactament en cap regla predefinida, el sistema cau en respostes genèriques o completament irrelevantes, demostrant que no té comprensió semàntica real. Aquestes eines no “entenen” el significat tot i que ens poden donar la sensació que si, és simplement un truc.

Punts clau de les limitacions:

- Rigidesa: Una variació mínima (sinònims, ordre de paraules) activa la regla per defecte.
- No comprensió real: Només *pattern matching*, sense processament semàntic.
- Problemes d'escalabilitat: calen regles infinites per cobrir totes les possibilitats.
- Sense context: no recorda converses anteriors ni manté la coherència.

Els primers passos per comprendre els texts

NLTK (Natural Language Toolkit, 2001) és la primera gran llibreria Python per a NLP. Ideal per aprendre conceptes pas a pas: transparent, didàctica, molt documentada. Perfecta per entendre com funcionen les coses per dins.

spaCy (2015) va arribar amb un enfocament completament diferent: velocitat industrial, models entrenats amb Machine Learning i un pipeline integrat llest per a producció. Podríem dir que és el OpenCV del NLP.

En aquesta secció veurem els mateixos conceptes amb les dues llibreries costat a costat, per entendre quan usar cadascuna i per què spaCy s'ha convertit en l'estàndard industrial.

```
[2]: !python -m spacy download es_core_news_sm
```

```
Collecting es-core-news-sm==3.8.0
  Downloading https://github.com/explosion/spacy-
models/releases/download/es_core_news_sm-3.8.0/es_core_news_sm-3.8.0-py3-none-
any.whl (12.9 MB)
12.9/12.9 MB
36.0 MB/s eta 0:00:00
Installing collected packages: es-core-news-sm
Successfully installed es-core-news-sm-3.8.0
Download and installation successful
You can now load the package via spacy.load('es_core_news_sm')
Restart to reload dependencies
If you are in a Jupyter or Colab notebook, you may need to restart Python in
order to load all the package's dependencies. You can do this by selecting the
'Restart kernel' or 'Restart runtime' option.
```

```
[3]: !python -m spacy download ca_core_news_sm
```

```
Collecting ca-core-news-sm==3.8.0
  Downloading https://github.com/explosion/spacy-
models/releases/download/ca_core_news_sm-3.8.0/ca_core_news_sm-3.8.0-py3-none-
any.whl (19.6 MB)
19.6/19.6 MB
33.2 MB/s eta 0:00:00
Installing collected packages: ca-core-news-sm
Successfully installed ca-core-news-sm-3.8.0
```

Download and installation successful

You can now load the package via `spacy.load('ca_core_news_sm')`

Restart to reload dependencies

If you are in a Jupyter or Colab notebook, you may need to restart Python in order to load all the package's dependencies. You can do this by selecting the 'Restart kernel' or 'Restart runtime' option.

```
[4]: # SETUP: NLTK + spaCy
import os
import nltk, spacy
import matplotlib.pyplot as plt
import numpy as np
from collections import Counter, defaultdict
from spacy import displacy

nltk.download('punkt', quiet=True)
nltk.download('stopwords', quiet=True)
nltk.download('punkt_tab', quiet=True)
nltk.download('averaged_perceptron_tagger', quiet=True)

#nltk_data_dir = '/root/nltk_data'
#os.makedirs(nltk_data_dir, exist_ok=True)

#nltk.download('punkt_tab', download_dir=nltk_data_dir)
#nltk.data.path.append(nltk_data_dir)

# NLTK: descargar recursos
#for recurs in ['punkt', 'punkt_tab', 'stopwords',
#               ↪ 'averaged_perceptron_tagger', 'wordnet']:
#    nltk.download(recurs, quiet=True, download_dir=nltk_data_dir)
#nltk.data.path.append(nltk_data_dir)

from nltk.tokenize import word_tokenize, sent_tokenize, TweetTokenizer
from nltk.corpus import stopwords
from nltk.stem import SnowballStemmer, PorterStemmer
from nltk.probability import FreqDist
from nltk.util import ngrams

# spaCy: cargar modelos
nlp = spacy.load('es_core_news_sm')
nlp_ca = spacy.load('ca_core_news_sm')

print(f"NLTK {nltk.__version__}")
print(f"spaCy {spacy.__version__} - pipeline: {' → '.join(nlp.pipe_names)}")
```

NLTK 3.9.1

spaCy 3.8.11 - pipeline: tok2vec → morphologizer → parser → attribute_ruler → lemmatizer → ner

El pipeline de spaCy executa TOT en una sola crida `nlp(text)`:

tokenitzar → POS → morfologia → dependències → NER

Però NLTK requereix cridar cada eina per separat i de vegades amb ajuda “manual”.

###Tokenització

La tokenització és el primer pas obligatori de qualsevol sistema NLP perquè converteix el text brut (una cadena caòtica de caràcters) en unitats manejables (tokens) que els algorismes poden processar.

Cas especial	Text d'entrada	Sortida esperada	Per què importa
Paraules	“Hola,món”	[“Hola”, “món”]	Separar sense espais
Puntuació	“Hola!”	[“Hola”, “!”]	No enganxar a paraules
Apostrofs català	“l'àvia”	[“l'”, “àvia”]	Respectar contraccions
Nombres	“2026”	[“2026”]	Unitat semàntica
Acrònims	“NASA”	[“NASA”]	No dividir majúscules
Emojis	“ ”	[“ ”]	Llenguatge modern
URLs	“https://ex.com”	[“https://ex.com”]	Unitat funcional

La tokenització va més enllà de “separar en paraules”. És dividir el text en unitats semàntiques amb significat independent.

```
[5]: text = "Yann LeCun diu: 'Deep Learning és la clau del futur intel·ligent!'\n      ↪(visita https://yann.lecun.com)"

# llibreria no especialitzada .split()
print(f"split():          {text.split()}\n")

#NLTK
# separació en tokens ("paraules") - la versió en català dóna problemes
print(f"word_tokenize:    {word_tokenize(text, language='spanish')}\n")

# separació en frases - la versió en català dóna problemes
text_m = "Bon dia! Com estàs? Saluda al Dr. Yann LeCun de part meva."
print(f"sentence_tokenize: {sentence_tokenize(text_m, language='spanish')}\n")

#SpaCy
spacy_toks = [tok.text for tok in nlp(text) if not tok.is_space]
print(f"\nspacy word_tokenize:      {spacy_toks}")

doc_m = nlp(text_m)
frases_spacy = list(doc_m.sents)
print(f"spacy sentence_tokenize:      {frases_spacy}")

split():          ['Yann', 'LeCun', 'diu:', "'Deep', 'Learning', 'és', 'la',
'clau', 'del', 'futur', "intel·ligent!", '(visita', 'https://yann.lecun.com)']
```

```
word_tokenize: ['Yann', 'LeCun', 'diu', ':', '"Deep', 'Learning', 'és', 'la',
'clau', 'del', 'futur', 'intel·ligent', '!', '"', '(', 'visita', 'https', ':',
'//yann.lecun.com', ')']
```

```
sent_tokenize: ['Bon dia!', 'Com estàs?', 'Saluda al Dr. Yann LeCun de part
meva.']
```

```
spaCy word_tokenize: ['Yann', 'LeCun', 'diu', ':', '"', 'Deep',
'Learning', 'és', 'la', 'clau', 'del', 'futur', 'intel·ligent', '!', '"', '(',
'visita', 'https://yann.lecun.com', ')']
spaCy sent_tokenize: [Bon dia! Com estàs?, Saluda al Dr. Yann LeCun de
part meva.]
```

###Stop Words i Neteja

Stop words: paraules buides (“el”, “és”, “un”, “molt”, “que”, “i”) que apareixen constantment però aporten zero informació semàntica.

```
[6]: text_sw = "Python és el millor llenguatge del món i tots ho sabeu"

sw_nltk = set(stopwords.words('spanish')) #català falla
sw_spacy = nlp.Defaults.stop_words

print(f"Stop words NLTK (spanish): {len(sw_nltk)}")
print(f"Stop words spaCy (model sm): {len(sw_spacy)}")

# NLTK: comparació de string contra la llista
toks_nltk_sw = word_tokenize(text_sw.lower(), language='spanish')
nets_nltk = [t for t in toks_nltk_sw if t not in sw_nltk and t.isalpha() and
↳ len(t) > 2]

# spaCy: atribut tok.is_stop calculat pel model
doc_sw = nlp(text_sw)
nets_spacy = [tok.text.lower() for tok in doc_sw if not tok.is_stop and tok.
↳ is_alpha and len(tok.text) > 2]

print(f"\nNLTK ({len(nets_nltk)} nets): {nets_nltk}")
print(f"spaCy ({len(nets_spacy)} nets): {nets_spacy}")
```

Stop words NLTK (spanish): 313

Stop words spaCy (model sm): 521

NLTK (6 nets): ['python', 'millor', 'llenguatge', 'món', 'tots', 'sabeu']

spaCy (6 nets): ['python', 'millor', 'llenguatge', 'món', 'tots', 'sabeu']

```
[7]: text = "Python és molt popular per intel·ligència artificial i machine learning.
↳"

#TOTS els mots (amb stops)
```

```

tots = [t.lower() for t in word_tokenize(text) if t.isalpha()]
freq_tots = Counter(tots)

#SENSE stops (només mots rellevants)
sw = set(stopwords.words('spanish'))
nets = [t for t in tots if t not in sw and len(t) > 2]
freq_nets = Counter(nets)

print("TOP 5 AMB stops: ", freq_tots.most_common(5))
print("TOP 5 SENSE stops:", freq_nets.most_common(5))

```

```

TOP 5 AMB stops: [('python', 1), ('és', 1), ('molt', 1), ('popular', 1),
('per', 1)]
TOP 5 SENSE stops: [('python', 1), ('molt', 1), ('popular', 1), ('per', 1),
('artificial', 1)]

```

Les stop words són mots buits com “és”, “molt”, “per”, “i” o “del” que, tot i ocupar un 70-80% del text, no aporten significat rellevant per l'anàlisi.

Quan les eliminem, els conceptes importants com “Python” o “artificial” passen al capdavant, permetent que els models de NLP es concentrin en el contingut real en lloc de processar soroll irrellevant.

##Entra en joc Machine Learning: llibreria spaCy

spaCy fa el salt a Machine Learning real. En lloc de regles escrites a mà, utilitza models estadístics entrenats sobre milions de textos.

Una de les grans diferències és el seu *pipeline* automatitzat. Amb NLTK has d'enconcatar diverses línies de codi per a qualsevol tasca que spaCy ho pot resoldre en una sola línia.

Podríem dir que spaCy és l'equivalent de OpenCV per a NLP.

Models oficials spaCy (2026) per a català/castellà:

Idioma	Model petit	Model mitjà	Model gran
Castellà	es_core_news_sm	es_core_news_md	es_core_news_lg
Català	ca_core_news_sm	ca_core_news_md	ca_core_news_lg

Aquest codi demostra el poder de spaCy, ja que amb `doc = nlp(text)` realitza 5 tasques NLP simultàniament i extreu 7 atributs intel·ligents del token.

```

[8]: nlp = spacy.load("ca_core_news_sm")

text = "La professora de l'optativa de Machine Learning explica NLP als
      ↪estudiants de l'Institut TIC de Barcelona."
doc = nlp(text)

tok = doc[1] # 'professora'
print(f"tok.text      → '{tok.text}'")
print(f"tok.lemma_    → '{tok.lemma_}'")

```

```

print(f"tok.pos_      → '{tok.pos_}'")
print(f"tok.dep_      → '{tok.dep_}'")
print(f"tok.is_stop   → {tok.is_stop}")
print(f"tok.head.text → '{tok.head.text}'")

```

```

tok.text      → 'professora'
tok.lemma_    → 'professor'
tok.pos_      → 'NOUN'
tok.dep_      → 'nsubj'
tok.is_stop   → False
tok.head.text → 'explica'

```

###POS Tagging + Lemmatization

spaCy assigna etiquetes gramaticals (POS tags) a cada token utilitzant Universal Dependencies, un estàndard global de 17 categories que funciona per qualsevol idioma.

```

NOUN  Noms: casa, aplicació, Python
VERB  Verbs: crear, explicar, funciona
ADJ   Adjectius: intel·ligent, ràpid
PROPN Noms propis: Barcelona
DET   Articles: el, la, un (stop words)
ADP   Preposicions: de, a, per
AUX   Auxiliar : va, ha, és
PUNCT Puntuació : . , !

```

L'ús d'una xarxa neuronal CNN permet mirat el context complet de la frase.

```

[9]: import pandas as pd

text = "La professora suplent de l'optativa de Machine Learning explica NLP als
      estudiants de l'Institut TIC de Barcelona."
doc = nlp(text)

df = pd.DataFrame([
    {'Text': tok.text, 'Lema': tok.lemma_, 'POS': tok.pos_, 'Stop': tok.is_stop}
    for tok in doc if not tok.is_space
])

print(df.to_string(index=False))

print(f"NOMS:      {'', '.join([t.lemma_.lower() for t in doc if t.pos_=='NOUN'
    and not t.is_stop])}")
print(f"VERBS:     {'', '.join([t.lemma_.lower() for t in doc if t.pos_=='VERB'
    and not t.is_stop])}")
print(f"ADJECTIUS: {'', '.join([t.lemma_.lower() for t in doc if t.pos_=='ADJ'
    and not t.is_stop])}")
print(f"NOMS PROPRIIS: {'', '.join([t.text
    for t in doc if t.
    pos_=='PROPN'])}")

```


Text	Lema	POS	Stop
La	el	DET	True
professora	professor	NOUN	False
suplent	suplent	ADJ	False
de	de	ADP	True
l'	el	DET	False
optativa	optativa	NOUN	False
de	de	ADP	True
Machine	Machine	PROP	False
Learning	Learning	PROP	False
explica	explicar	VERB	False
NLP	NLP	PROP	False
a	a	ADP	True
ls	el	DET	False
estudiants	estudiant	NOUN	False
de	de	ADP	True
l'	el	DET	False
Institut	Institut	PROP	False
TIC	TIC	PROP	False
de	de	ADP	True
Barcelona	Barcelona	PROP	False
.	.	PUNCT	False

NOMS: professor, optativa, estudiant
VERBS: explicar
ADJECTIUS: suplent
NOMS PROPRIIS: Machine, Learning, NLP, Institut, TIC, Barcelona

Anàlisi Sintàctic

```
[10]: text = "La professora suplent de l'optativa de Machine Learning explica NLP als_
      ↪estudiants de l'Institut TIC de Barcelona."
doc = nlp(text)

for tok in doc:
    if tok.is_punct or tok.is_space: continue
    arrel = "ARREL" if tok.dep_=='ROOT' else ""
    cap = tok.head.text if tok.dep_!='ROOT' else ""
    print(f" {tok.text:<16} {tok.pos_:<8} {tok.dep_:<12} {cap}{arrel}")
```

La	DET	det	professora
professora	NOUN	nsubj	explica
suplent	ADJ	amod	professora
de	ADP	case	optativa
l'	DET	det	optativa
optativa	NOUN	nmod	professora
de	ADP	case	Machine
Machine	PROP	nmod	optativa
Learning	PROP	flat	Machine

explica	VERB	ROOT	ARREL
NLP	PROPN	obj	explica
a	ADP	case	estudiants
ls	DET	det	estudiants
estudiants	NOUN	obl	explica
de	ADP	case	Institut
l'	DET	det	Institut
Institut	PROPN	nmod	estudiants
TIC	PROPN	flat	Institut
de	ADP	case	Barcelona
Barcelona	PROPN	flat	Institut

Fem la representació visual de l'anàlisi sintàctic:

```
[11]: from spacy import displacy
      from IPython.core.display import display, HTML

      displacy.render(doc, style='dep', jupyter=True)
```

<IPython.core.display.HTML object>

A partir d'aquí podem realitzar l'extracció de SVO (Subjecte-Verb-Objecte):

```
[12]: doc = nlp("La professora suplent explica NLP als estudiants de Barcelona.")

      for tok in doc:
          if tok.dep_ == 'ROOT': # Troba verb principal
              subj = next((c.text for c in tok.children if c.dep_ == 'nsubj'), ' ') #
              ↳ Subjecte
              obj = next((c.text for c in tok.children if c.dep_ in ('dobj',
              ↳ 'obj')), ' ') # Objecte
              print(f"[{subj}] → {tok.lemma_} → [{obj}]"
```

[professora] → explicar → [NLP]

L'anàlisi sintàctica amb spaCy estructura el text per crear grups de dades relacionades útils en projectes professionals:

- Gràfics de coneixement que connecten persones i accions.
- Resum de textos extreient idees principals
- Sistemes de preguntes i respostes que troben qui fa què
- Control de contingut detectant relacions sospitoses.
- Anàlisi de sentiments que entenen el context de cada opinió.

###**Detecció d'Entitats (NER)** És una tècnica de Processament de Llenguatge Natural (NLP) que identifica i classifica automàticament les entitats importants dins d'un text.

Exemples de tipus d'entitats:

PER	Persona:	Núria Pujol
ORG	Organització:	Google, Generalitat
LOC	Lloc:	Barcelona, San Francisco

DATE Data: 20 de novembre, el 2024
MONEY Diners: 500 milions d'euros
CARDINAL Número: 2.000, tres milions

Per limitacions model català ca_core_news_sm només reconeix PERSONES/LLOCS/ORG bàsics.

```
[13]: text_ner = "La Núria Pujol explica NLP als estudiants de l'Institut TIC de  
      ↳Barcelona durant el 23 de febrer de 2026 per poc més de 2.000€ al mes."  
  
      doc_ner = nlp(text_ner.strip())  
  
      for ent in doc_ner.ents:  
          print(f"   {ent.text:<28} {ent.label_}")
```

Núria Pujol	PER
NLP	ORG
Institut TIC	ORG
Barcelona	LOC

És millor fer una representació:

```
[14]: colors_ent = {  
      'PER': '#ff7043',       # Taronja  
      'ORG': '#42a5f5',       # Blau  
      'LOC': '#66bb6a',       # Verd  
      'DATE': '#ffa726',       # Amber  
      'MONEY': '#26c6da',      # Blau cyan  
      'CARDINAL': '#ef5350',  # Vermell  
}  
  
      options = {  
          'colors': colors_ent,  
          'ents': ['PER', 'ORG', 'LOC', 'DATE', 'MONEY', 'CARDINAL'],  
          'compact': True,  
          'bg': '#fafafa',  
          'font': 'Source Sans Pro'  
      }  
  
      displacy.render(doc_ner, style='ent', options={'colors': colors_ent})
```

<IPython.core.display.HTML object>

Word2Vec Word2Vec (Google, 2013) revoluciona NLP passant de “comptar paraules” a “mesurar similituds semàntiques” mitjançant embeddings vectorials.

Els mètodes anteriors com one-hot encoding fallaven perquè tractaven totes les paraules com a igualment distants: “gat” és tan lluny de “gos” com de “cotxe”, tot i compartir contextos. Word2Vec resol això i fa que les paraules que apareixen en contextos similars tenen significats similars, per tant els seus vectors han de ser propers en un espai vectorial.

Word2Vec transforma cada paraula en un vector dens de 100-300 dimensions après entrenar una

xarxa neuronal simple sobre un gran corpus.

El resultat és la propietat màgica dels embeddings: operacions vectorials com:

`vec("rei") - vec("home") + vec("dona")    vec("reina")    vec(Paris) - vec(França) + vec(Espanya)    vec(Madrid)`

Els passos són: * Tokenitzar corpus en frases * Definir finestra contextual (window=4), 3) * Entrenar amb Gensim (sg=1 per corpus petits), 4) * Validar amb similituds i operacions vectorials

###**Paraules com a vectors**

```
[15]: !pip install gensim
```

```
Collecting gensim
  Downloading gensim-4.4.0-cp312-cp312-
manylinux_2_24_x86_64.manylinux_2_28_x86_64.whl.metadata (8.4 kB)
Requirement already satisfied: numpy>=1.18.5 in /usr/local/lib/python3.12/dist-
packages (from gensim) (2.0.2)
Requirement already satisfied: scipy>=1.7.0 in /usr/local/lib/python3.12/dist-
packages (from gensim) (1.16.3)
Requirement already satisfied: smart_open>=1.8.1 in
/usr/local/lib/python3.12/dist-packages (from gensim) (7.5.0)
Requirement already satisfied: wrapt in /usr/local/lib/python3.12/dist-packages
(from smart_open>=1.8.1->gensim) (2.1.1)
Downloading
gensim-4.4.0-cp312-cp312-manylinux_2_24_x86_64.manylinux_2_28_x86_64.whl (27.9
MB)
27.9/27.9 MB
76.3 MB/s eta 0:00:00
Installing collected packages: gensim
Successfully installed gensim-4.4.0
```

```
[16]: import gensim.downloader as api
model = api.load('glove-wiki-gigaword-50') # model preentrenat
```

```
[=====] 100.0% 66.0/66.0MB
downloaded
```

```
[17]: print("Similituds pre-entrenades:")
print(f"cat-dog: {model.similarity('cat', 'dog'):.3f}")
print(f"cat-car: {model.similarity('cat', 'car'):.3f}")
print(f"python-java: {model.similarity('python', 'java'):.3f}")
print(f"king-queen: {model.similarity('king', 'queen'):.3f}")
```

```
Similituds pre-entrenades:
cat-dog: 0.922
cat-car: 0.364
python-java: 0.477
king-queen: 0.784
```

S'ha d'entrenar perquè no existeixen models pre-entrenats de qualitat per català (tot i que necessita moltes dades):

```
[18]: from gensim.models import Word2Vec

corpus = [
    "el gat negre corre ràpid pel jardí",
    "el gos petit dorm a la cadira cada dia",
    "cotxe ha passat molt ràpid i no ha parat al semàfor",
    "python és un llenguatge de programació molt popular",
    "java és un altre llenguatge popular però mai superarà a python",
    "machine learning utilitza algorismes per aprendre patrons de les dades"
    # ... més frases
]

frases = [frase.lower().split() for frase in corpus]

model = Word2Vec(
    sentences = frases,
    vector_size = 100, # dimensions del vector
    window = 4, # finestra de context: ±4 paraules
    min_count = 1,
    sg = 1, # skip-gram (millor per corpus petits)
    epochs = 500,
    seed = 42
)

print(f"Vocabulari: {len(model.wv)} paraules")
print(f"Dimensions: {model.vector_size}")

print("Similitud entre paraules:")
print(f"gat - gos: {model.wv.similarity('gat', 'gos')}")
print(f"gat - cotxe: {model.wv.similarity('gat', 'cotxe')}")
print(f"python - java: {model.wv.similarity('python', 'java')}")
```

Vocabulari: 45 paraules

Dimensions: 100

Similitud entre paraules:

gat - gos: 0.9938327670097351

gat - cotxe: 0.9951670169830322

python - java: 0.997745931148529

En els següents exemples utilitzarem l'anglès, perquè com hem vist, en català funciona una mica pitjor.

Word2Vec necessita milers de frases per aprendre vectors significatius i Gensim inclou el corpus text8 (17M paraules d'Wikipedia en anglès) que permet demostrar les propietats reals dels embeddings en pocs segons. En castellà o català necessitaríem descarregar un corpus així de gran extern.

```
[19]: # Corpus text8 d'Wikipedia (17M paraules)
```

```
corpus = api.load('text8')
sentences = list(corpus)
```

```
[=====] 100.0% 31.6/31.6MB
downloaded
```

```
[22]: # Entrenament
```

```
model = Word2Vec(
    sentences = sentences,
    vector_size = 100,      # dimensions del vector
    window = 5,            # context: ±5 paraules
    min_count = 10,         # ignora paraules molt rares
    sg = 1,                # skip-gram (millor per corpus grans)
    workers = 4,           # paral·lelisme (un per nucli de CPU)
    epochs = 5,            # passades completes sobre el corpus
    seed = 42
)
```

```
print(f" Vocabulari: {len(model.wv):,} paraules")
```

```
print(f" Dimensions: {model.vector_size}")
```

```
print("Primeres 10 dimensions del vector de 'computer' de 100:")
```

```
vec = model.wv['computer']
```

```
print(f" [{', '.join([f'{v:.3f}' for v in vec[:10]])}], ...")
```

```
Vocabulari: 47,134 paraules
```

```
Dimensions: 100
```

```
Primeres 10 dimensions del vector de 'computer' de 100:
```

```
[0.192, -0.021, 0.403, 0.007, -0.063, 0.245, 0.253, 0.061, 0.040, 0.542, ...]
```

Càlcul de similitud a través del cosinus

Per què el cosinus i no la distància?

Imagina dos vectors:

$v1 = [1, 1]$ (modul petit)

$v2 = [10, 10]$ (modul gran, però mateixa direcció)

La distància euclidiana seria gran (~12.7), però apunten al mateix lloc en l'espai semàntic. La similitud del cosinus mesura l'angle entre vectors, ignorant el mòdul:

$$\cos() = \frac{v1 \cdot v2}{|v1| \times |v2|}$$

$\cos() = 1.0$ idèntics (angle 0°)

$\cos() = 0.9$ molt similars

$\cos() = 0.5$ moderadament relacionats

`cos() = 0.0` independents (angle 90°)
`cos() < 0` significats oposats (angle > 90°)

Veguem un exemple de com obtenir paraules similars:

```
[23]: words = ['computer', 'internet', 'software',  
              'king', 'queen', 'prince',  
              'cat', 'dog', 'rabbit']  
  
for word in words:  
    similars = model.wv.most_similar(word, topn=5)  
    top3 = ', '.join([f"'{w}' ({s:.2f})" for w, s in similars])  
    print(f"  '{word}' → {top3}")  
  
  'computer' → 'computers'(0.80), 'computing'(0.78), 'hardware'(0.77),  
'bootstrap'(0.75), 'microcontroller'(0.73)  
  'internet' → 'dsl'(0.77), 'usenet'(0.77), 'bitnet'(0.76), 'uucp'(0.74),  
'isps'(0.74)  
  'software' → 'developers'(0.82), 'adware'(0.81), 'linux'(0.80),  
'customization'(0.79), 'freeware'(0.79)  
  'king' → 'prince'(0.77), 'canute'(0.74), 'haakon'(0.73), 'vii'(0.72),  
'valdemar'(0.72)  
  'queen' → 'elizabeth'(0.81), 'highness'(0.77), 'regnant'(0.75),  
'consort'(0.73), 'margrethe'(0.73)  
  'prince' → 'regent'(0.78), 'king'(0.77), 'braganza'(0.76), 'pretender'(0.76),  
'highness'(0.75)  
  'cat' → 'eared'(0.72), 'felis'(0.70), 'silky'(0.70), 'albino'(0.70),  
'hyena'(0.70)  
  'dog' → 'hound'(0.75), 'badger'(0.74), 'dogs'(0.74), 'keeshond'(0.73),  
'dachshund'(0.73)  
  'rabbit' → 'beanstalk'(0.75), 'powerpuff'(0.74), 'beast'(0.73), 'porky'(0.73),  
'lumberjack'(0.72)
```

Observa que ens les paraules morfològicament relacionades (computer/computers, dog/dogs) apareixen com a molt similars. El model no coneix gramàtica, però ha arribat a aquesta conclusió a través del context.

Aritmètica vectorial

Mikolov i el seu equip van descobrir que els vectors apresos permeten càlcul semàntic:

```
vec("king") - vec("man") + vec("woman")    vec("queen")
```

```
[24]: analogies = [  
    ("king", "woman", ["man"], "king - man + woman = ? (esperem: queen)"),  
    ("france", "berlin", ["paris"], "france - paris + berlin = ? (esperem: ↳  
↳germany)"),  
    ("good", "bad", ["better"], "good - better + bad = ? (esperem: ↳  
↳worse)"),
```

```

(["walk", "swim"], ["walked"], "walk - walked + swim = ? (esperem:
↳swam)"),
(["actor", "woman"], ["man"], "actor - man + woman = ? (esperem:
↳actress)"),
]

for pos, neg, desc in analogies:
    result = model.wv.most_similar(positive=pos, negative=neg, topn=1)
    print(f"{desc} → {result[0][0]} ({result[0][1]:.2f})")

```

```

king - man + woman = ? (esperem: queen) → queen (0.66)
france - paris + berlin = ? (esperem: germany) → germany (0.86)
good - better + bad = ? (esperem: worse) → luck (0.57)
walk - walked + swim = ? (esperem: swam) → greyhound (0.55)
actor - man + woman = ? (esperem: actress) → actress (0.84)

```

Les analogies no funcionen sempre perfectament. Depenen de la qualitat i mida del corpus. Amb text8 (petit), alguns resultats seran incorrectes. Amb Google News (100B paraules), la precisió puja molt.

El model és tan bo com les dades que ha vist.

Visualització de Word2Vec

Cada paraula té un vector de 100 nombres que captura el seu significat basant-se en el context on apareix al corpus de Wikipedia. Per veure-ho, fem dues proves visuals:

- **Heatmap de similituds:** Prenem 12 paraules dividides en 4 temes (animals, països, tecnologia, reialesa). Calculem quina similitud tenen entre elles (de 0 a 1). Si Word2Vec funciona, els animals s'haurien d'assemblar molt entre ells (cat-dog ~0.8), els països també (França-Espanya ~0.8), però un animal i un país seran molt diferents (cat-França ~0.2). Això es veu com blocs brillants a la diagonal del heatmap —cada tema forma el seu propi grup compacte.
- **Mapa 2D (PCA):** Com no podem veure 100 dimensions, les comprimim a 2D. Si funciona, cada tema apareix agrupat per color: tots els animals junts en vermell, tots els països junts en blau, etc. És la prova geomètrica que Word2Vec no només compta paraules sinó que les organitza per significat conceptual.

```

[25]: import numpy as np
import matplotlib.pyplot as plt
from sklearn.decomposition import PCA

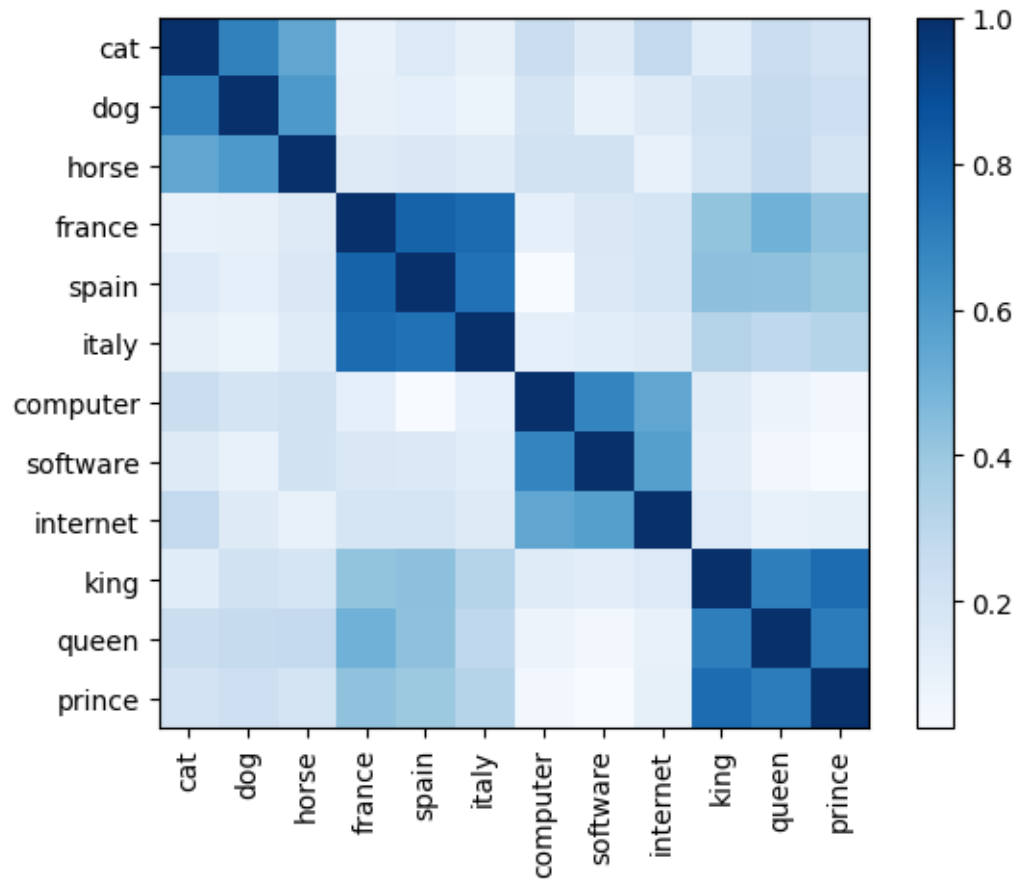
# 4 grups × 3 paraules = 12 paraules simples
paraules = ['cat', 'dog', 'horse', 'france', 'spain', 'italy',
            'computer', 'software', 'internet', 'king', 'queen', 'prince']

# 1. HEATMAP similituds (12×12)
mat = np.array([[model.wv.similarity(p1,p2) for p2 in paraules] for p1 in
↳paraules])

```

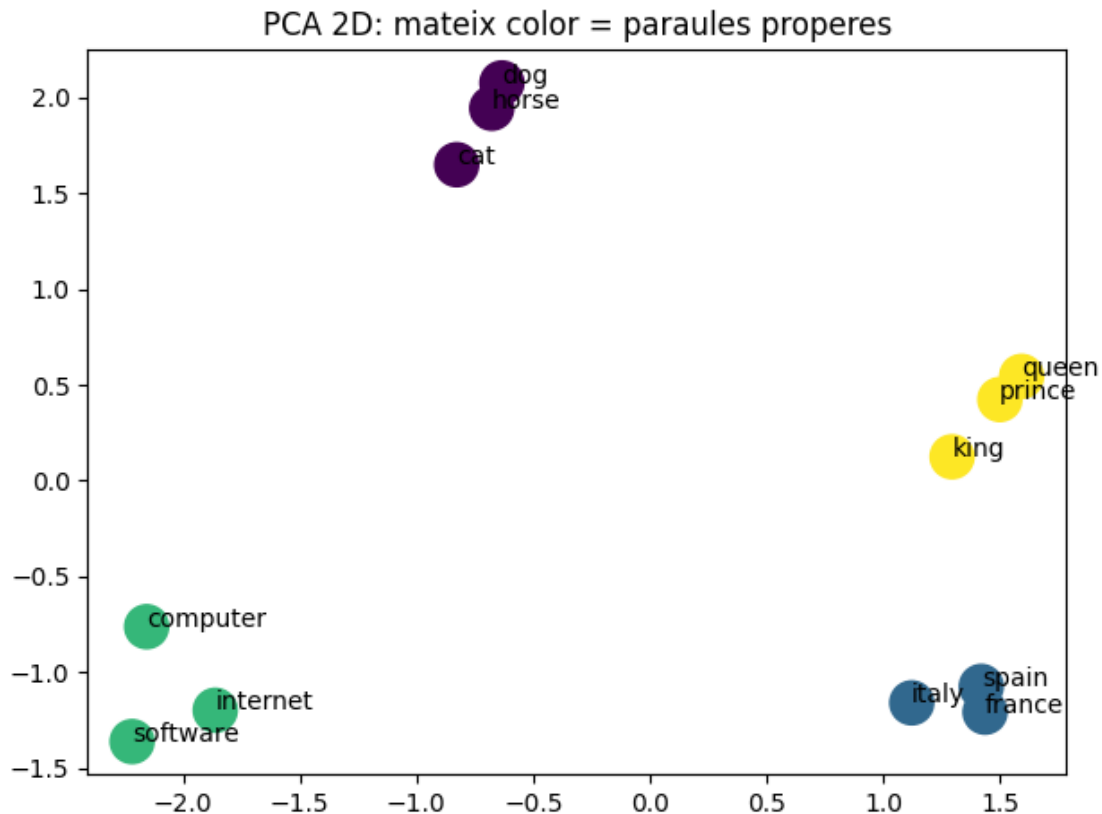


```
plt.imshow(mat, cmap='Blues'); plt.colorbar(); plt.xticks(range(12), paraules,
    ↪rotation=90); plt.yticks(range(12), paraules)
plt.show()
```



```
[26]: vectors = np.array([model.wv[p] for p in paraules])
      coords = PCA(n_components=2).fit_transform(vectors)

      colors = [0,0,0, 1,1,1, 2,2,2, 3,3,3] # 3 animals, 3 països, etc
      plt.scatter(coords[:,0], coords[:,1], c=colors, s=300)
      for i, p in enumerate(paraules): plt.annotate(p, coords[i])
      plt.title('PCA 2D: mateix color = paraules properes')
      plt.tight_layout()
      plt.show()
```



Cerca semàntica

La cerca tradicional és lèxica: busca coincidències exactes de text. Si l'usuari escriu “cotxe”, no troba documents que parlin d’ “automòbil” o “vehicle”. Word2Vec permet una cerca per significat.

Veguem quins sinònims ens troba Word2Vec donat el corpus de Wikipedia.

```
[27]: casos = [
    ("python", "Programació o serp?"),
    ("bank", "Financer o riu?"),
    ("science", "Camp científic"),
    ("music", "Música relacionada")
]

for paraula, desc in casos:
    print(f"{desc}")
    similars = model.wv.most_similar(paraula, topn=5)
    print(f"'{paraula}' → {[f'{w}({s:.2f})' for w,s in similars]}")
```

Programació o serp?

'python' → ['monty(0.85)', 'gilliam(0.67)', 'pythons(0.67)', 'grail(0.64)', 'circus(0.62)']

Financer o riu?

```
'bank' → ['banks(0.75)', 'monetary(0.72)', 'ecb(0.65)', 'fund(0.64)',
'cemas(0.63)']
Camp científic
'science' → ['fiction(0.76)', 'multidisciplinary(0.73)', 'bioethics(0.70)',
'criminology(0.70)', 'actuarial(0.69)']
Música relacionada
'music' → ['folk(0.81)', 'jazz(0.80)', 'musical(0.79)', 'reggae(0.79)',
'electronica(0.78)']
```

Limitació fonamental de Word2Vec: assigna un sol vector fix per paraula, independentment del context. Exemple: “bank” té el mateix vector per “el bank ofereix préstecs” (financer) i “passen per el bank del riu” (geogràfic). Les paraules amb múltiples significats (polysemy) es “barregen” en una representació única que no distingeix sentit específic.

BERT (Google 2018) resol tot això amb vectors contextuais dinàmics (depenen del context). Cada paraula té un vector únic segons el seu context complet gràcies a Transformers bidireccions.

Word2Vec (2013) → BERT (2018) → LLMs (2020+)

Similitud semàntica amb spaCy

Integra vectors preentrenats de 300 dimensions directament al pipeline NLP complet (tokenització + POS + NER + dependències + similituds), mentre Word2Vec amb Gensim requereix que l'usuari descarregui/entreni el seu propi model sobre corpus específic.

A diferència de Word2Vec (que treballa paraula a paraula), spaCy pot calcular la similitud entre frases senceres fent la mitjana dels vectors de cada token:

```
[28]: !python -m spacy download ca_core_news_md
```

```
Collecting ca-core-news-md==3.8.0
  Downloading https://github.com/explosion/spacy-
models/releases/download/ca_core_news_md-3.8.0/ca_core_news_md-3.8.0-py3-none-
any.whl (49.2 MB)
49.2/49.2 MB
14.3 MB/s eta 0:00:00
Installing collected packages: ca-core-news-md
Successfully installed ca-core-news-md-3.8.0
Download and installation successful
You can now load the package via spacy.load('ca_core_news_md')
Restart to reload dependencies
If you are in a Jupyter or Colab notebook, you may need to restart Python in
order to load all the package's dependencies. You can do this by selecting the
'Restart kernel' or 'Restart runtime' option.
```

```
[29]: import spacy

nlp_md = spacy.load("ca_core_news_md")
```

```
[30]: parells = [
        ("M'agrada programar Python", "Gaudeixo codificant Python"),
```

```

    ("El ML usa dades per aprendre", "Els algorismes d'IA aprenen d'exemples"),
    ("El cel està ennuvolat", "Programar Python és divertit"),
    ("L'estudiant DAM va fer una app", "L'alumne d'informàtica va desenvolupar_
↪una aplicació")
]

for a, b in parells:
    sim = nlp_md(a).similarity(nlp_md(b))
    print(f"'{a}' vs '{b}' → {sim:.3f}")

```

```

'M'agrada programar Python' vs 'Gaudeixo codificant Python' → 0.746
'El ML usa dades per aprendre' vs 'Els algorismes d'IA aprenen d'exemples' →
0.350
'El cel està ennuvolat' vs 'Programar Python és divertit' → 0.250
'L'estudiant DAM va fer una app' vs 'L'alumne d'informàtica va desenvolupar una
aplicació' → 0.852

```

Limitació important: spaCy (com Word2Vec) calcula la similitud de la frase com a mitjana dels seus vectors. Frases com “El gat va mossegar el gos” i “El gos va mossegar el gat” tindran similitud 1, malgrat tenir significats oposats.

Per a comprensió real del significat de les frases cal un model transformer com BERT o RoBERTa.